A PROJECT REPORT

on

Meru-Prastāra as Pascal Triangle and Dynamic Programming Model

Submitted By

Nayan Borse

in partial fulfillment for the award of the

degree of

BACHELOR OF SCIENCE

in

COMPUTER SCIENCE

Under the guidance of

Asst. Prof. Sini.V.

Department of Computer Science & Information Technology



SMT. DEVKIBA MOHANSINHJI CHAUHAN COLLEGE OF COMMERCE AND SCIENCE

SILVASSA-UT OF DD & DNH-396230

Sem V

2026-27

Smt. Devkiba Mohansinhji Chauhan College of Commerce and Science
(Affiliated to University of Mumbai)
SILVASSA-UT OF DD & DNH-396230

Department of Computer Science & Information Technology

CERTIFICATE

This is to certify that **Nayan Borse** of **T.Y.Bsc.CS** **(Sem V)** class has satisfactorily completed the Project **"Meru-Prastāra as Pascal Triangle and Dynamic Programming Model"** to be submitted in the partial fulfillment for the award of **Bachelor of Science** in **Computer Science** during the academic year 2026-2027.

Date of Submission:

Project Guide **Signature of Examiner**

College Seal **Coordinator**

DECLARATION

We, **Nayan Borse** hereby declare that the project entitled **"Meru-Prastāra as Pascal Triangle and Dynamic Programming Model"** Submitted in the partial fulfillment for the award of **Bachelor of Science** in **Computer Science** during the academic year **2026-27** is my original work and the project has not formed the basis for the award of any degree, associateship, fellowship or any other similar titles.

Signature of the Student:

Place:

Date:

ACKNOWLEDGEMENT

We would like to convey our heartfelt gratitude to **Asst. Prof. Sini.V** for her tremendous direction and assistance in the completion of our project. We are eternally grateful to you.

We are deeply indebted to Department Coordinator **Asst. Prof. Sini.V** and our Principal **Dr. Seema Pillai** for giving us this valuable opportunity to do this project.

We are thankful to our college **Smt. Devkiba Mohansinhji Chauhan College of Commerce and Science** for considering our project and extending help at all stages needed during our work of collecting information regarding this project.

We express our hearty thanks to them for their assistance without which it would have been difficult in finishing this project synopsis and project review successfully.

We also like to thank my friends and parents for their support and encouragement as we worked on this assignment.

Nayan Borse

INDEX

Sr. No.	Particulars	Page No.
1	Title Page	1
2	Problem Statement	2
3	Conceptual Mapping Table	3
4	Algorithm Spacification	4
5	Working Code	5
6	Test Cases	6
7	Complexity & Limitations	7
8	Conclusion	8

Introduction

Meru-Prastāra as Pascal Triangle and Dynamic Programming Model

Meru-Prastāra is an ancient Indian mathematical arrangement that can be studied in relation to **Pascal's Triangle** and modern **Dynamic Programming**. The project focuses on understanding the structural relationship between Meru-Prastāra and Pascal's Triangle and representing this relationship using a computational model.

The project demonstrates how the values in the triangle can be generated using a **recursive relationship**, where each internal value is obtained from the sum of the two values directly above it. This provides a simple example of how a mathematical concept can be represented through an algorithm and implemented using programming techniques.

The project also connects this mathematical structure with **Dynamic Programming**, particularly the idea of solving a problem by using previously computed values instead of repeatedly calculating the same values. This makes the model useful for understanding concepts such as **recursion, algorithms, arrays, computation, and dynamic programming**.

Problem Statement

Meru-Prastāra is an ancient Indian mathematical arrangement associated with the study of combinatorial coefficients and prosody. Its structure is closely related to what is known today as Pascal's Triangle. Each inner value is obtained by adding two values from the previous row. This makes Meru-Prastāra an excellent example for explaining dynamic programming in Computer Science.

The problem is to design and implement a program that generates the first n rows of Meru-Prastāra, where every row begins and ends with 1 and each internal value is calculated from the two adjacent values in the previous row. The implementation should demonstrate how a mathematical pattern can be represented as a reusable computational model using dynamic programming.

The program should accept a non-negative integer n and produce the corresponding triangle. It should also handle invalid input appropriately and be tested with at least ten different test cases.

Conceptual Mapping Table

IKS / Mathematical Concept	Pascal Triangle Interpretation	Computer Science Concept	Mapping
Meru-Prastāra	Triangular arrangement of coefficients	Data structure / matrix-like representation	The mathematical triangle is stored as a list of rows.
First and last element	Every row starts and ends with 1	Base case	Boundary values provide the starting conditions for each row.
Inner element	Sum of two upper adjacent elements	Recurrence relation	A value is computed from previously calculated values.
Previous rows	Earlier coefficients are reused	Dynamic programming	Previously computed results are stored and reused.
n-th row	Contains binomial coefficients	State / subproblem	Each row represents one stage of the computation.
Combination coefficient	C(n,k)	Algorithmic computation	The program calculates combinatorial values through a recurrence instead of repeated factorial work.
Triangular structure	Rows grow by one element	Nested lists / 2D structure	The output can be represented as a list containing variable-length rows.
Recurrence	C(n,k)=C(n-1,k-1)+C(n-1,k)	DP transition	The next state depends on two already solved states.

Example of Meru-Prastāra / Pascal Triangle:

```
                1
              1   1
            1   2   1
          1   3   3   1
        1   4   6   4   1
      1   5  10  10   5   1
```

The central computational rule is that each internal entry is the sum of the two entries directly above it. This local dependency is the key connection to dynamic programming.

Algorithm Specification (Pseudo-code)

Algorithm: Generate Meru-Prastāra using Dynamic Programming

```
ALGORITHM GenerateMeruPrastara(n)

INPUT: n = number of rows
OUTPUT: Meru-Prastāra containing n rows

1. If n < 0:
       Report "Invalid input"
       Stop

2. Create an empty list triangle

3. For i = 0 to n - 1:
       Create a row containing (i + 1) elements initialized to 1

       For j = 1 to i - 1:
           row[j] = triangle[i - 1][j - 1] + triangle[i - 1][j]

       Append row to triangle

4. Return triangle

END ALGORITHM
```

Working principle:

For every row, the first and last positions remain 1. Only the internal positions are calculated. For example, when generating the row [1, 4, 6, 4, 1], the internal values are obtained as 1+3=4, 3+3=6, and 3+1=4 from the previous row [1, 3, 3, 1].

Working Code

Python implementation:

```
def generate_meru_prastara(n):
    """Generate n rows of Meru-Prastara (Pascal Triangle)."""
    if not isinstance(n, int) or isinstance(n, bool):
        raise TypeError("n must be an integer")

    if n < 0:
        raise ValueError("n must be non-negative")

    triangle = []

    for i in range(n):
        row = [1] * (i + 1)

        for j in range(1, i):
            row[j] = triangle[i - 1][j - 1] + triangle[i - 1][j]

        triangle.append(row)

    return triangle


def print_meru_prastara(n):
    triangle = generate_meru_prastara(n)

    if n == 0:
        print("(empty triangle)")
        return

    width = len(" ".join(map(str, triangle[-1])))

    for row in triangle:
        text = " ".join(map(str, row))
        print(text.center(width))


# Example
n = 6
print_meru_prastara(n)
```

Expected output for n = 6:

```
          1
        1   1
      1   2   1
    1   3   3   1
  1   4   6   4   1
1   5  10  10   5   1
```

Minimum 10 Test Cases

The following test cases verify normal inputs, boundary cases, larger inputs, and invalid inputs.

Test	Input	Expected Result	Purpose	Status
1	0	[]	Zero rows / boundary case	Pass
2	1	[[1]]	Single row	Pass
3	2	[[1], [1,1]]	Smallest non-trivial triangle	Pass
4	3	[[1], [1,1], [1,2,1]]	Verify first internal coefficient	Pass
5	4	Last row = [1,3,3,1]	Verify row recurrence	Pass
6	5	Last row = [1,4,6,4,1]	Verify common example	Pass
7	6	Last row = [1,5,10,10,5,1]	Verify larger coefficients	Pass
8	10	10 rows generated; last row has 10 values	Medium input	Pass
9	15	15 rows generated; last row has 15 values	Larger input	Pass
10	20	20 rows generated; last row has 20 values	Performance sanity test	Pass
11	-1	ValueError	Reject negative input	Pass
12	3.5	TypeError	Reject non-integer input	Pass

Test-case validation examples:

```
generate_meru_prastara(3)
# [[1], [1, 1], [1, 2, 1]]

generate_meru_prastara(5)
# [[1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1]]

generate_meru_prastara(0)
# []

generate_meru_prastara(-1)
# Raises ValueError

generate_meru_prastara(3.5)
# Raises TypeError
```

Complexity & Limitations

Time Complexity

For n rows, the total number of elements generated is $1 + 2 + ... + n = n(n+1)/2$. Therefore, the time complexity is $O(n^2)$. Each internal coefficient is calculated once.

Space Complexity

The complete triangle is stored in memory, so the space complexity is $O(n^2)$. If only the latest row is required, the space can be reduced to $O(n)$.

Limitations

- The complete triangle requires quadratic memory, which becomes large for very high values of n.
- Python integers can grow to many digits for large rows, increasing both memory usage and computation time.
- The program demonstrates the recurrence clearly, but it does not cover every possible use of Meru-Prastāra in ancient Indian prosody.
- The program assumes that the input represents the number of rows rather than a specific combinatorial query.
- The centered text output is intended for small and medium examples; very large triangles are not convenient to display in a terminal.

Conclusion

Meru-Prastāra provides a strong example of how an Indian mathematical idea can be connected to modern Computer Science. Its triangular arrangement corresponds closely to Pascal's Triangle, and its rule of obtaining each internal value from two values in the previous row naturally forms a dynamic programming recurrence.

The implemented Python program stores previously computed rows and reuses them to construct new rows. This avoids repeatedly calculating the same intermediate values and demonstrates an important principle of dynamic programming: solve smaller subproblems once, store their results, and use those results to solve larger problems.

Thus, the study of Meru-Prastāra is not only historically and mathematically significant, but also useful for understanding data structures, recurrence relations, algorithm design, dynamic programming, and computational efficiency. This mapping demonstrates how traditional knowledge can be interpreted through a modern Computer Science perspective.

References / Suggested Reading

- Indian Knowledge Systems (IKS) – introductory material on traditional Indian mathematics and prosody.
- Standard discrete mathematics references covering binomial coefficients and Pascal's Triangle.
- Standard algorithms and dynamic programming textbooks for recurrence relations and complexity analysis.